This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

Explainable GNN-based Approach to Fault Forecasting in Cloud Service Debugging

Dániel Unyi†, Ernő Rigó*, Bálint Gyires-Tóth† and Róbert Lovas*
†Department of Telecommunications and Artificial Intelligence, Faculty of Electrical Engineering and Informatics, Budapest University of Technology and Economics, Műegyetem rkp. 3., H-1111 Budapest, Hungary.
*Institute for Computer Science and Control (SZTAKI), Hungarian Research Network (HUN-REN)

***Abstract*—Debugging cloud services is increasingly challenging due to their distributed, dynamic, and scalable nature. Traditional methods struggle to handle large state spaces and the complex interactions between microservices, making it difficult to diagnose failures and identify critical components. This paper presents a Graph Neural Network (GNN)-based approach that enhances cloud service debugging by predicting system-level fault probabilities and providing interpretable insights into failure propagation. Our method models microservice interactions as graphs, where failures propagate probabilistically. Using Markov Decision Processes (MDPs), we simulate failure behaviors, capturing the probabilistic dependencies that influence system reliability. The trained GNN not only predicts fault probabilities but also identifies the most failure-prone microservices and explains their impact. We evaluate our approach on various service mesh structures, including feature-enriched, tree-structured, and general directed acyclic graph (DAG) architectures. Results indicate that our method is effective in the operational phase of cloud services, enabling proactive debugging and targeted optimization. This work represents a step toward more interpretable, reliable, and maintainable cloud infrastructures.**

***Index Terms*—Cloud computing, Software debugging, Microservice architectures, Deep learning, Graph neural networks, Explainable AI, Fault prediction**

I. Introduction

Cloud computing's distributed, dynamic, and scalable nature makes debugging cloud services increasingly complex. Traditional debugging, monitoring, and troubleshooting methods struggle to capture the dependencies between microservices. One of the greatest challenges in debugging is not just detecting failures, but understanding why they occur and which components are responsible. Debugging tools must move beyond simple anomaly detection and provide interpretable insights that highlight the root causes of failures and the most influential microservices affecting system reliability. To ensure high availability and reliability, there is a critical need for explainable methods that analyze, predict, and clarify cloud service behavior. To bridge this gap, our work integrates failure propagation modeling with GNN-based explainability, which allows both fault probability estimation and critical component identification within service meshes.

R. Lovas and E. Rigó are with the Institute for Computer Science and Control (SZTAKI), Hungarian Research Network (HUN-REN), and with the John von Neumann Faculty of Informatics, Óbuda University, Hungary, email: {robert.lovas, erno.rigo}@sztaki.hun-ren.hu

D. Unyi and B. Gyires-Tóth are with the Department of Telecommunications and Artifical Intelligence, Faculty of Electrical Engineering and Informatics, Budapest University of Technology and Economics, email:{unyi.daniel, toth.b}@tmit.bme.hu

B. Gyires-Tóth and R. Lovas contributed equally to this work and share senior authorship.

A. Cloud Computing with Reference Architecture and Service Mesh Concepts

Cloud computing has transformed modern application architectures by enabling scalable, on-demand resource provisioning over the internet. As cloud-native applications continue to expand, microservices architectures [1] have become the de facto standard for ensuring scalability, availability, and manageability. However, the heterogeneous and distributed nature of microservices introduces challenges such as efficient orchestration, secure communication, and observability. Managing these interactions becomes particularly difficult as the number of microservices grows, increasing the complexity of debugging and performance monitoring. To address these issues, the service mesh paradigm has emerged as a key solution.

A service mesh is a dedicated infrastructure layer that manages service-to-service communication, ensuring transparent, reliable, and secure interactions between microservices. It facilitates load balancing, service discovery, traffic management, failure recovery, and monitoring, significantly reducing the overhead associated with microservices orchestration. By decoupling service communication from application logic, service meshes simplify infrastructure management and enhance observability. Complementing this, reference architectures [2]–[6] serve as standardized blueprints for cloud service design, incorporating best practices, interoperability guidelines, and reliability principles to improve system resilience.

Leading service mesh implementations—Istio, Linkerd, and AWS App Mesh [7]—integrate reference architecture principles to facilitate microservice communication and management. For example, Istio provides advanced traffic management, security policies, and telemetry collection, establishing a strong foundation for operational control and observability. However, while these tools enhance visibility into service interactions, analyzing failure propagation across microservices remains a challenge, requiring additional techniques to assess system reliability and performance.

In summary, service meshes and reference architectures improve operational efficiency, security, and observability, but effectively analyzing service behavior still requires additional

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

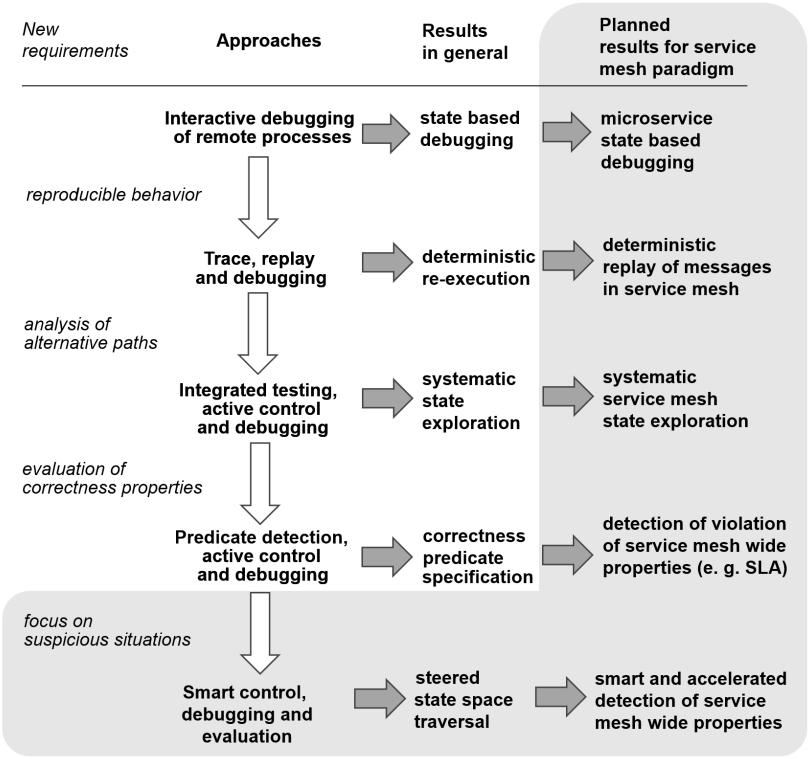


Fig. 1. Classification of general distributed debugging methodologies [8] extended with our planned results in service mesh environment (grey background).

tools that can track dependencies and evaluate system performance under failure conditions.

B. Debugging Challenges

Debugging service mesh architectures can be challenging due to their complexity and distributed nature [9]. A primary issue is observability, which refers to analyzing system behavior and inter-service communication. In practice, only partial observability is often possible due to factors such as black-box (proprietary) services or probe effects, where monitoring tools can influence system behavior. This incomplete view makes it difficult to determine which microservices contribute most to system failures and how faults propagate through the mesh.

Beyond observability, identifying and diagnosing faults in large-scale service meshes is particularly challenging. Configuration errors, misrouted traffic, and performance bottlenecks can cause latency spikes, service degradation, and system faults [10]. However, tracing these issues back to their root cause is difficult because service meshes abstract communication details, making failure propagation paths non-trivial to reconstruct. Debugging inter-service failures is further complicated by dynamic workloads and ephemeral service instances, where failures may only manifest intermittently or under specific operating conditions.

Additionally, failures in service meshes are often difficult to reproduce, making root cause analysis highly reactive rather than proactive. Without effective replay mechanisms, debugging relies on post-mortem analysis, where engineers must manually analyze logs and traces, often with limited visibility into how failures emerged. This lack of systematic failure reproduction hinders efforts to anticipate and mitigate faults before they escalate.

C. Theoretical Background and Research Plan

Distributed debugging methodologies can be classified based on their support for global predicate specification and detection, as well as their ability to identify root causes of software bugs in distributed systems [8]. Over time, these methodologies have evolved to address the limitations of previous approaches. Figure 1 summarizes these methods and highlights our planned contributions in service mesh environments (grey background).

At the most basic level, interactive debugging extends traditional sequential debugging to remote processes, allowing real-time monitoring but lacking insight into inter-process interactions, making it unsuitable for distributed systems. To overcome this, the "*Trace, replay and debugging*" approach (Figure 1, row 2) records selected execution traces to enable deterministic reproduction of faulty behaviors under controlled conditions.

Building on this, the "*Integrated testing, active control and debugging*" (Figure 1, row 3) introduces mechanisms that steer execution along predefined computation paths, improving reproducibility beyond passive observation. A more advanced technique, "*Predicate detection, active control and debugging*" (Figure 1, row 4), allows users to define correctness conditions (predicates) that are continuously evaluated during execution, enhancing systematic failure detection.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

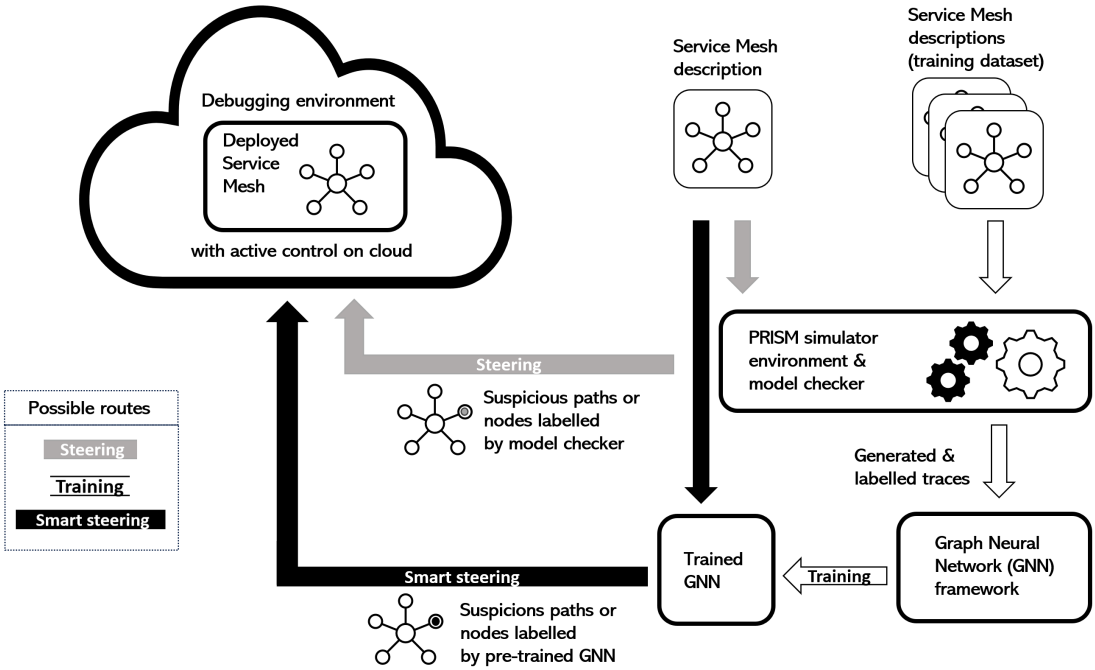


Fig. 2. Proposed framework for debugging cycle with tools and dataflow.

One of our previous works extended this methodology to the deployment phase of orchestrated cloud services [11], focusing on improving reliability (including platform-agnostic behavior) in complex deployment strategies. This approach proved useful in detecting timing or architecture-dependent failures within deployment descriptions and orchestration steps.

This research further narrows the scope to developing a smart steering mechanism within the "Smart Control, Debugging, and Evaluation" paradigm (Figure 1, row 5). In our previous work [12], we applied the PRISM simulator environment with model checker, along with autoencoders, Long Short-Term Memory (LSTM) networks, and Graph Neural Networks (GNNs), to analyze system behavior. Extending beyond this, in this paper we focus on leveraging GNNs for fault prediction and explainability to enhance debugging in cloud service meshes.

D. Proposed Debugging Framework

This paper shifts the focus from debugging during the deployment phase to debugging during operation, where failures often arise due to dynamic interactions between microservices. Our approach introduces explainability into debugging, using Graph Neural Networks (GNNs) to not only predict system-level fault probabilities but also to provide interpretable insights into failure propagation and root causes (see black route in Figure 2). Unlike deterministic replay-based debugging, which relies on replicating failure states, our method uses observed data to improve fault localization and guide debugging efforts.

Our approach builds on the service mesh paradigm, which provides a structured, policy-driven environment for managing microservices. Service mesh descriptors, such as those in Istio, define critical configurations for network traffic management (e.g., routing rules, load balancing), security policies (e.g., authentication, authorization), and observability features (e.g., telemetry, logging). These descriptors allow fine-grained control over service behavior without modifying application code, making them an ideal foundation for our debugging framework (Figure 2, "Service Mesh Description").

The "*Predicate detection, active control, and debugging*" approach is embedded within the "*Steering*"-related components of our framework (gray route in Figure 2). The '*PRISM simulator environment with model checker*' detects suspicious paths or nodes and labels them to steer debugging efforts toward critical areas. In our previous work [12], we developed a simplified service mesh model, representing RESTful service nodes as a dependency tree. This was extended into a REST modeling framework using a domain-specific language (DSL) to define probabilistic service interactions, allowing the generation of probabilistic state machines for each microservice based on its strategy and network connections. Our PRISM-based model generator translates these DSL descriptions into Markov Decision Processes (MDPs) by generating module descriptions in PRISM for all nodes and their dependencies. Once validated, RESTful communication flows of arbitrary length can be simulated. Additionally, PRISM performs formal verification, ensuring the model satisfies specified correctness properties using probabilistic model checking. This allows us to define temporal logic properties and systematically analyze failure cases, identifying deadlocks, inconsistencies, and failure probabilities [13] (see 'Generated & labeled traces' in Fig. 2).

In this paper, we propose a debugging framework that integrates Graph Neural Networks (GNNs) within the '*Training*'-related components (white route in Figure 2). Our approach (see Section IV) addresses the challenge of debugging complex

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

service meshes by leveraging fault probability prediction and GNN-based explainability to identify weak components (see Section IV-C). Instead of relying solely on deterministic replay or exhaustive model checking, our method learns from observed system behavior to prioritize high-risk execution paths and potential failure states. Key components include:

- Training a GNN to predict system-level fault probabilities, using service mesh graphs where nodes represent microservices and edges capture inter-service interactions.
- GNN-based explainability to identify critical components, selecting a minimal but informative subgraph that preserves predictive accuracy.
- Automated generation, augmentation, and labeling of training data using the PRISM simulator/model checker, which allows the GNN to learn from probabilistic execution traces (white route in Figure 2, see also Section III).
- Combining Markov Decision Process (MDP) modeling with GNN-based learning, where MDPs encode service interaction dynamics, and GNNs refine failure prediction and debugging automation.

The suspicious situations detected by pre-trained GNNs may realize '*Smart steering*' (black route in Figure 2), allowing cloud debugging tools [11], [14] to proactively identify and localize failures. This process can bypass the PRISM simulator environment and model checker, as the trained GNN can directly analyze service mesh descriptions. Our experimental results (see Sections V and VI) demonstrate the effectiveness of GNN-based explainability in detecting critical components and operating under limited observability conditions.

II. Related Works

With the increasing complexity of cloud architectures, maintaining system stability and fault detection has become a critical challenge. Continuous monitoring collects logs on memory issues, server failures, and application performance [15], but the vast volume of logs makes it difficult to promptly isolate faults and take proactive measures before system failures occur. Moreover, failure datasets are typically much smaller than complete system logs, making fault analysis challenging.

Farooq et al. define faults as abnormal conditions that reduce or eliminate a component's ability to function correctly, with failures emerging as their observable consequences [16]. Identifying root causes before failures manifest is crucial for proactive debugging.

Traditional formal verification methods have been applied to analyze cloud architectures, particularly in service-oriented environments. Zatout et al. [17] propose transaction modeling for improving reliability in RESTful applications, while Souri et al. [18] review service composition verification techniques. These studies highlight the importance of formal methods in ensuring the robustness of RESTful systems.

Although formal methods provide strong guarantees, their applicability to large-scale, dynamic service meshes is limited due to computational cost and reliance on predefined models. In order to address these limitations, data-driven fault detection methods have emerged, using statistical analysis and machine learning to improve fault diagnosis and prediction. The following subsections discuss both traditional and ML-based fault detection approaches, with a particular focus on GNN-based methods for service meshes.

A. Traditional Fault Detection Methods

Rule-based fault detection methods, such as intrusion and anomaly detection systems, monitor network or host activity for deviations from expected behavior [19]. These methods rely on either signature-based detection, using predefined anomaly patterns, or anomaly-based detection, identifying statistical deviations from normal system behavior. Abbasi et al. [20] propose an anomaly detection system for cloud data centers that enhances fault resilience through real-time monitoring and behavioral analysis.

Beyond anomaly detection, automated testing and resilience strategies have been explored for service meshes. Karn et al. [21] introduce a framework that monitors microservice communication traces, using component fault injectors to simulate delays, traffic restrictions, and terminations. Their approach detects faulty connections via dashboards and mitigates issues through scaling, failover, and circuit breakers, offering a reactive mechanism for service reliability.

Formal modeling techniques have also been applied to fault analysis in cloud environments. Cao and Niu [22] apply higher-order Markov graphs to analyze cloud deployments and big data logs, identifying complex failure patterns. However, their approach lacks an active control mechanism to steer the debugging process toward critical failure states.

Pontillo et al. [23] propose FERRARI, a failure reproduction tool that generates test cases and analyzes stack traces to identify faults. FERRARI includes a similarity scoring mechanism to estimate how closely generated test cases match real failure conditions. While their tool aids in failure reproduction, it does not incorporate ML-guided steering as in our work.

B. Machine Learning Based Fault Detection Methods

Machine learning-based fault detection methods generally fall into supervised and unsupervised categories. Supervised models learn to distinguish between normal and faulty behavior using labeled input-output pairs, making them effective when labeled data is available. In contrast, unsupervised approaches detect anomalies by identifying deviations from normal patterns without requiring explicit fault labels, allowing them to adapt to previously unseen failure modes.

Zhang et al. [24] propose an online fault detection model for cloud systems using a support vector machine (SVM) with automated parameter tuning (SVM-Grid). Their approach improves fault prediction by selecting relevant system attributes via Pearson correlation and principal component analysis (PCA). While effective, it focuses on general cloud fault detection and does not explicitly model inter-service dependencies within service meshes.

Zhou et al. [25] introduce MEPFL, a machine learning-based method for latent error prediction and fault localization in microservice applications. It leverages system trace logs to train models at both the trace and microservice levels, allowing

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

TABLE I
COMPARISON OF SELECTED RELATED WORKS. (ML: MACHINE LEARNING)

reference	architecture level	approach	applied methods	focus area	fault injection
Smara [19]	component	rule-based	signature/anomaly detection	component safety	manual
Abbasi [20]	network level	rule-based	behavioral analysis	network anomaly detection	manual
Karn [21]	service mesh	automated testing	fault injection, rule-based analysis	communication reliability	semi-automated
Cao & Niu [22]	cloud service	stochastic model	Higher-order Markov Graph	fault analysis in cloud logs	N/A
Pontillo [23]	service mesh	automated testing	test-case generation, similarity scoring	failure reproduction	N/A
Zhang [24]	system	ML (supervised)	SVM + PCA + Pearson correlation	general fault detection	N/A
Zhou [25]	microservice-level	ML (supervised)	log analysis, trace classification	latent error prediction	semi-automated
Meng [26]	service mesh	ML + RL	reinforcement learning, CTMC	resilience strategy optimization	semi-automated
Rouf [27]	service mesh	ML	GNN + GRU + permutation testing	failure prediction + RCA	N/A
Maroudis [28]	client-service network	GNN	graph-based SLA violation prediction	SLA prediction	N/A
Wu [29]	microservice-level	GNN	anomaly tracing, root cause localization	performance diagnosis	N/A
Wang [30]	cross-network system	GNN + EVT	topological causal discovery	fault localization	N/A
Gan [31]	microservices	unsup. GNN	trace-level root cause analysis	debugging	N/A
Lovas [11], [12]	multi-level	ML, GNN	autoencoder, LSTM, GNN, MDP	generic debugging	automated
This paper	service mesh	GNN + XAI	MDP-based failure simulation	critical component identification	automated

real-time fault detection and classification. By analyzing execution traces and incorporating fault injection during training, MEPFL improves failure diagnosis in dynamic production environments. However, it primarily focuses on predicting fault occurrences rather than modeling how failures propagate within a service mesh.

Meng et al. [26] present a reinforcement learning approach to predict fault resilience behaviors in service meshes. Their method optimizes service-level configurations that influence request-response behavior under failure conditions, which enables adaptive fault resilience strategies across both individual services and multi-service orchestrations. While effective for mitigating faults, this approach emphasizes policy optimization rather than failure probability estimation or interpretable fault propagation modeling.

Rouf et al. [27] propose InstantOps, a multi-modal failure prediction and root cause analysis (RCA) framework that integrates logs, metrics, traces, and microservices events. It constructs a dynamic service call flow graph, where traces define the structure, while logs and resource metrics enrich node attributes. InstantOps applies GNNs to capture spatial dependencies and Gated Recurrent Units (GRUs) to model temporal failure patterns, enabling proactive failure prediction and precise root cause localization. Moreover, permutation testing is used to assess node importance within failure propagation chains. While InstantOps effectively fuses multi-modal observability data and models both spatial and temporal failure dynamics, it differs from our work, which explicitly learns a failure propagation model to identify critical subgraphs contributing to system faults.

C. Graph Neural Network Based Fault Detection

Maroudis et al. [28] investigate SLA violation prediction using GNNs, exploring time series, content-based, and context-based (graph-based) data representations. Their study demonstrates that graph-based representations significantly enhance prediction accuracy by capturing the associativity of clients and integrating them with traditional models. However, their approach is concerned with SLA violations rather than analyzing failure propagation or identifying structurally critical components in cloud services.

Wu et al. [29] introduces MicroRCA, an attributed graph model to trace anomaly propagation across services and machines in microservice architectures. It performs real-time root cause localization by correlating application performance symptoms with system resource utilization. While effective for diagnosing performance issues, its primary focus is on linking performance anomalies to resource bottlenecks, making it complementary to our work.

Wang et al. [30] applies hierarchical graph neural networks to discover intra-network and cross-network causal dependencies for fault localization. It combines topological causal discovery with an Extreme Value Theory (EVT)-based method to rank probable root causes. This approach is well-suited for analyzing multi-level causal interactions but does not explicitly account for the role of service dependencies in fault propagation.

Gan et al. [31] introduce a trace-based root cause analysis system, Sleuth, for large-scale microservices, using an unsupervised GNN to assess the causal impact of each span in a trace. It clusters traces using a trace distance metric to reduce the data volume required for root cause localization, with a strong emphasis on transferability across microservice applications. While Sleuth focuses on performance debugging, our work provides a complementary perspective by learning an explicit failure propagation model and identifying critical subgraphs responsible for system faults.

Table I provides a comparative overview of selected related works according to (i) the architectural level they target, (ii) the overall approach they apply, (iii) the primary methods applied, (iv) their main focus areas, and (v) the strategy used for fault injection. This comparative analysis is partially informed by the survey presented in [32].

Our previous work [12] introduced a preliminary GNN-based fault prediction model for service meshes, leveraging a simplified RESTful service model with a dependency tree structure. This model was integrated with a domain-specific language (DSL) to define probabilistic service interactions, enabling the generation of probabilistic state machines for each microservice. Fault propagation dynamics were captured using Markov Decision Processes (MDPs), and the framework was validated using formal verification techniques. Even though this preliminary work demonstrated the applicability of GNNs

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

for failure prediction, it was limited to tree structured service meshes and lacked a dedicated explainability mechanism for identifying the most critical service components contributing to failures.

Building on this foundation, our current work generalizes the approach by extending the tree-structured service mesh model to feature-enriched, directed acyclic graph (DAG) service meshes. Unlike the earlier model, the new one supports node-level features such as failure probability (for leaf nodes) and retry count (for intermediary nodes). These enhancements offer a more realistic representation of service interactions and failure dependencies.

Our updated approach integrates GNN-based explainability to identify critical subgraphs and failure-prone microservices. The identified critical subgraphs provide hints for active steering of a cloud debugger tool towards the suspicious situations (component faults), allowing proactive fault localization and mitigation.

III. Modelling Service Meshes for Cloud Reference Architectures

A. Framework

Current modelling efforts build upon our previous work focusing on probabilistic model checking applications, more specifically the PRISM model checker [13] is employed to describe, simulate and formally validate models expressed as Markov decision processes (MDPs).

During further developments of the model, the PRISM language is used to define parallel composition of $modules$ describing various behavioural patterns in different cloud application modules. These patterns are composed of various communication $actions$ also serving as synchronization points between the modules. Critical model parameters and various features of interests are identified by PRISM $labels$ to aid tracking of behaviour changes in the cloud application. Parametric generation of various experiments is based on $constants$ that provide a straightforward method to adjust behavioural properties of the simulated application context.

Our work has gradually shifted from constrained or unconstrained $path\ simulation$ applications towards formal $model\ checking$ against various properties described as probabilistic linear temporal logical expressions, such as PCTL [33]. Ultimately, these expressions were used to define end-user observable features of the simulated cloud application, most prominently service availability or error rate provided by the user interface.

B. Faults in a Cloud Application

Errors in a cloud application can result in either transient or permanent disruption of their services. From the perspective of end-users these disruptions result in performance degradation or - in severe cases - functional or data losses.

Measurement efforts of cloud application quality aims to quantify these properties. In cases of formal service agreements, these quantitative metrics play a key role. Thresholds for these metrics, including the acceptable rate of system faults are often established in a Service Level Agreement (SLA) between the cloud service provider and the end-user. This kind of service measurement was first employed in IP networks [34]. The concept then was widely adopted by various sectors, including cloud applications [35].

Although transient errors or permanent disruptions that result in a performance degradation might play a major role in overall debugging of a cloud application, for the time of current research we are focusing on fatal service situations. For this purpose service level is defined as the probability of fatal service disruption (system fault) observed by the end-user. Contrary to the common practice of representing service availability as the percentage ratio of acceptable metrics values over time (e.g., requests are answered 99.995% within a specific period of time), we are representing fatal system disruption probability value (or in short, fault) as a standard probability value (e.g., requests are failing with a probability of 0.005 in the observed simulation window).

C. Service Mesh Architectures

In order to generalize and refine our models further leveraging on our previous achievements [12], we continued to investigate various methods of describing cloud use cases. Microservice-based architecture patterns, also known as service meshes, are becoming increasingly popular in modern cloud applications [36].

Cloud applications were chosen to be modelled as a set of loosely coupled web services. Intra-application integration is achieved by applying various services of the mesh service component. From these integration services, message interception and routing is observed to be most feasible to aid in simulation of RESTful cloud application services [37].

Our previous modelling efforts were aiming towards simplified representation of a RESTful service model by organizing application components by their dependency relationships into a directed acyclic graph (DAG), where the root node R is representing the user facing service component of the application. In the model, R is continuously triggering requests to intermediary nodes n and leaf nodes m. Requests are routed in a nondeterministic way from R through various number of intermediaries (n) ultimately arriving to a leaf m.

Two constants are introduced on the graph level to better describe application behaviour P_{fail} is the probability of failure of any leaf node (m), while $retries$ is a parameter of intermediaries (n) representing number of request retransmission attempts before giving up on an error. Responses of the application are originating from leaf nodes (m) then propagated back through a series of intermediaries (n) to be ultimately aggregated at R enabling observation of application error rates.

Based on this approach, each user request is simulated as a single chain of requests and responses along a path $[R, n_1, n_2, ..., n_x, m]$, representing each complex request by forming a random depth-first search (DFS) path walk, complete with occasional back-traces as instructed by the $retries$ parameter. Figure 3 illustrates a RESTful service mesh application model based on a random DAG.

In the course of our current work, model-level constants P_{fail} and $retries$ were further enhanced to become local

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

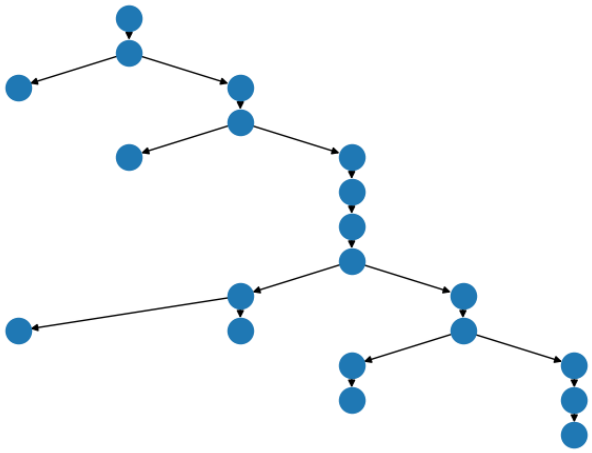


Fig. 3. Simple service mesh application topology

properties of leaf m and intermediary n nodes, becoming $P_{fail}[m]$ and $retries[n]$ respectively. This approach then was further enhanced by conducting a breadth-first walk on the DAG to group nodes by their distance d from the root node, thus forming node sets at different levels n_d of the DAG. New requests paths then were injected to the DAG in the form of edges (e) so that they only traverse forward the DAG, $\forall e_{new}(n_{d_1}, n_{d_2}) | d_1 < d_2$. Figure 4 illustrates the previous RESTful service application model with injected forward edges.

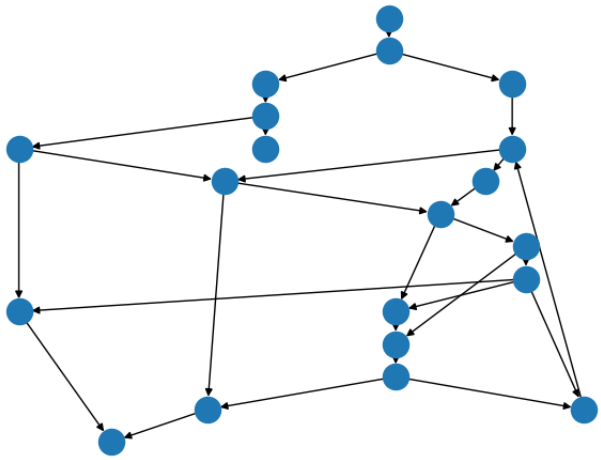


Fig. 4. Service mesh application topology with forward edges - modification of graph on Figure 3

It is also possible to calculate the probability of an overall application failure by checking the numerical value of the expression $P_{max} =?[F < TR_{failure}]$, where $R_{failure}$ is the "failure" state of the root node (R) and T is the number of model transitions measured from the initial state of "polling". Availability of the application for a given period of time T can also be considered a closely related property to P_{max}.

These enhancements further enhanced our simulation and training data generation capabilities, enabling modelling of RESTful cloud applications with increased complexity. New node-level parameters enabled clustering of previous experiments with subcases of topologically equivalent application models, but with distinct random values for $P_{fail}[m]$ and $retries[n]$.

A generic algorithm for dataset generation is supplied as pseudocode in Algorithm 1. Statements with $export$ keywords represent records output as part of the dataset. Statements with $generate$ and $filter$ keywords represent custom Python code by the authors, $generatePAs()$ is responsible for deriving probabilistic automatons (PAs) for nodes of a directed graph, $generatePRISM()$ will then translate this PA representation into PRISM source code. Statements with $random$ keyword annotate random data generation with uniform distribution. Finally, $pathSimulation$ and $checkFaultExpression$ represent external calls to the PRISM model checker.

Algorithm 1 pseudocode for dataset generation

```
for all cases do
    D := randomDAG(n_nodes)
    G := insertDFSEdges(D, p_down)
    for all subcases do
        for all n in G.nodes do
            if isLeaf(n) then
                n.P_fail := random(p_min, p_max)
            end if
            if isInternal(n) then
                n.retries := random(r_min, r_max)
            end if
        end for
        exportGEXF(id_subcase, G)
        S := generatePAs(G)
        P := generatePRISM(S)
        path := pathSimulation(P)
        exportPath(path)
        actions := filterActions(path)
        exportActions(actions)
        fault := checkFaultExpression(P)
        exportFault(id_subcase, fault)
    end for
    exportStatistics(subcases)
end for
exportStatistics(cases)
```

Algorithm 1 outlines a procedure for generating datasets that simulate RESTful service mesh applications using directed acyclic graphs (DAGs) and stochastic modeling tools like probabilistic automatons (PAs) and the PRISM model checker. The process begins with iterating over different application cases. For each case, a random DAG (D) is generated to represent the architecture of the application. Directed edges are then inserted into this graph to simulate potential request paths, guided by a probability (p_{down}) of establishing downward connections that respect the hierarchical structure of the DAG.

Within each subcase, nodes in the graph are classified based on their role: either as leaf nodes or intermediary nodes. Leaf nodes, which handle end-user requests, are assigned a failure probability (P_{fail}) generated randomly within defined bounds ((p_{min}, p_{max})). Intermediary nodes are configured with a

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

parameter denoting the number of retry attempts in case of failures, also determined randomly ((r_{min}, r_{max})).

The graph structure and its configurations are exported in GEXF format for interoperability and further analysis. The procedural transformation then involves deriving probabilistic automatons for the graph nodes, which are subsequently translated into the PRISM modeling language to facilitate formal verification and model checking of the stochastic models. A simulation of request paths is conducted, tracing the routes of requests across the service mesh to assess the pathways utilized.

Pathway data is exported for analysis, while actionable events within these paths are filtered for further inspection and exported separately. To ascertain the robustness of the subcases, a formal check is performed for expressions indicating system faults using the PRISM model checker, with the results recorded alongside the respective subcase identifiers.

The algorithm concludes by aggregating statistical data from individual subcases and cases, exporting these statistics to serve as a comprehensive dataset that encapsulates the behavior and properties of the simulated service mesh applications under varied conditions.

IV. Graph Neural Network for the Detection of Critical Components in Service Meshes

A. Graph Neural Networks

Graph Neural Networks (GNNs) have become the most common machine learning methods for predictive tasks within the domain of real-world networks. In these networks, each individual data point corresponds to a node, characterized not only by a feature vector x_i but also an adjacency vector a_i that specifies its connections to all other nodes. These feature vectors are organized into a feature matrix denoted as $\mathcal{X} \in \mathbb{R}^{N \times F}$, while the adjacency vectors take the form of an adjacency matrix $\mathcal{A} \in \mathbb{R}^{N \times N}$, where F and N respectively denote the number of features and the number of nodes in the network.

The tasks addressed by GNNs can be categorized into node-level, link-level, and graph-level tasks, depending on whether the labels pertain to nodes, links, or entire graphs. Given the sparsity of real-world networks, it is noteworthy that only a small number of entries in $\mathcal{A}$ contain non-zero values. To effectively handle such sparse matrices, GNNs incorporate so-called message passing layers [38], [39]. Message-passing layers are responsible for gathering information from neighboring nodes. This process can be mathematically expressed as:

$$x_i' = \phi \left(x_i, \bigoplus_{j \in \mathcal{N}(i)} \theta \left(x_i, x_j \right) \right)$$

In this expression, x_i is the feature vector associated with node i, and $\mathcal{N}(i)$ represents the set of neighboring nodes connected to node i. The symbol $\bigoplus$ represents a differentiable, permutation-invariant function (e.g., sum, mean, or max), while ϕ and θ denote differentiable functions, often realized as Multilayer Perceptrons (MLPs). In our study, we have adopted $\bigoplus = \sum$, and have configured ϕ and θ as $\{Linear\} \rightarrow \{PReLU\} \rightarrow \{Linear\}$.

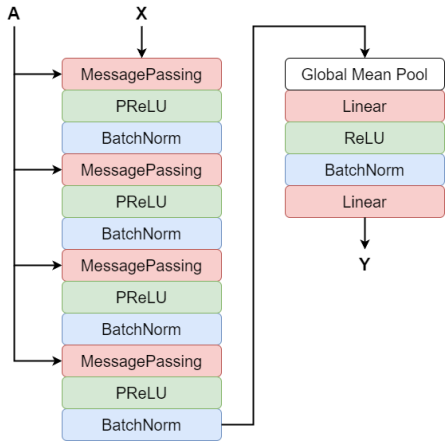


Fig. 5. The proposed GNN architecture for fault probability prediction.

B. GNN Training for Fault Prediction

To identify critical components within a service mesh, it is essential to have a strong baseline GNN model for fault probability prediction. With a well-trained model, we can assess how the removal of certain parts of the service mesh affects the system-level fault probability: a significant decrease indicates that those parts are weak components, while a significant increase suggests that those parts play a crucial role in maintaining the service mesh's performance.

Fault probability prediction is a graph-level regression task in which each service mesh is represented as a graph, and the target variable is the associated system-level fault probability. Each node within the service mesh graph is characterized by a single feature. For leaf nodes, this feature is the probability of failure, while for intermediary nodes, it represents the number of request retransmission attempts before giving up on an error. To facilitate modeling, we employed two distinct input channels: for leaves, the number of retries was set to 0 since they lack descendants, while for intermediaries, the probability of failure was set to 0 as these nodes do not directly experience failures. This approach ensures a consistent feature representation.

The dataset was randomly split into training, validation, and test sets, comprising 80%, 10%, and 10% of the service meshes, respectively. Our objective is to learn a function f that minimizes the mean squared error MSE$(f(\mathcal{G}), y)$ between the predicted system-level fault probabilities $f(\mathcal{G})$ and the ground-truth system-level fault probabilities y for the input graphs $\mathcal{G} = (\mathcal{X}, \mathcal{A})$.

To achieve this, we use a standard graph neural network architecture composed of the following layers: $\{MPBlock\} \rightarrow \{MPBlock\} \rightarrow \{MPBlock\} \rightarrow \{MPBlock\} \rightarrow \{GlobalMeanPool\} \rightarrow \{Linear\} \rightarrow \{ReLU\} \rightarrow \{Linear\}$ where each message passing block includes the following operations: $\{MessagePassing\} \rightarrow \{BatchNorm\} \rightarrow \{PReLU\}$. The complete architecture is illustrated in Figure 5.

We conducted meticulous hyperparameter tuning for our GNN model. The dimensionality of each of the 4 hidden layers were set to 32. The neural network was trained for 200 epochs

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

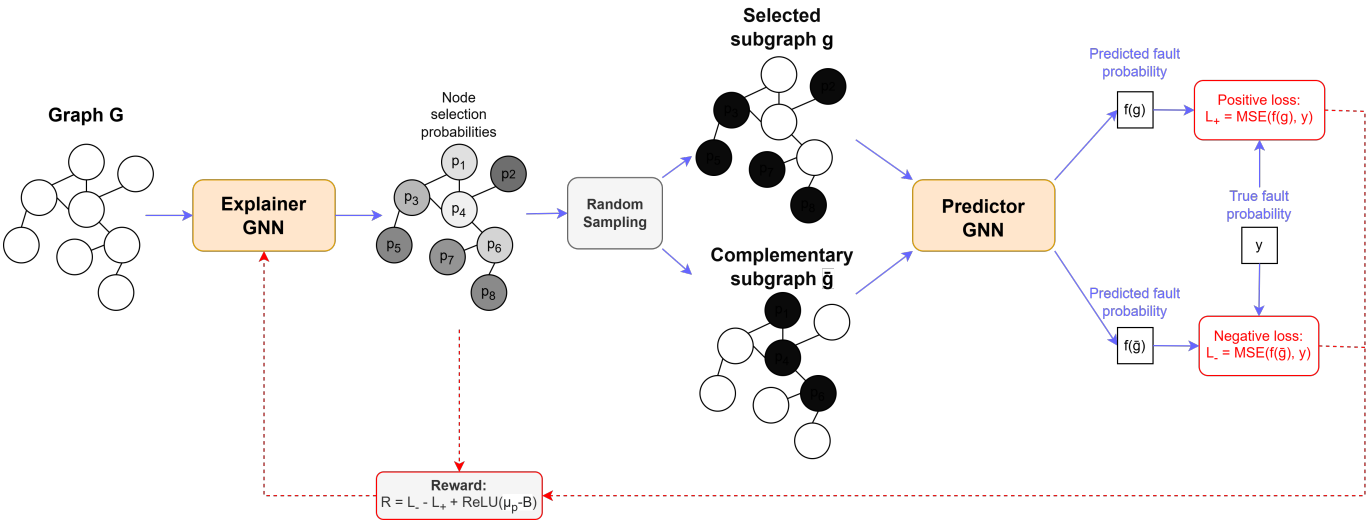


Fig. 6. Illustration of training the explainer GNN for identifying critical components in the service mesh. The explainer GNN generates probabilities for node masking, producing a mask that selects the most important nodes for fault probability prediction. The predictor GNN then evaluates the selected subgraph g and its complementary $\bar{g}$. If the mask captures the most relevant nodes, the predictor's performance on the subgraph g should closely match its performance on the original graph, while its performance on the complementary subgraph $\bar{g}$ should drop significantly, confirming that the selected nodes are necessary and sufficient for the prediction. The mean of the probabilities, μ_p, and the budget B (e.g., $B = 33\%$ means one-third of the nodes are retained) control the number of nodes selected.

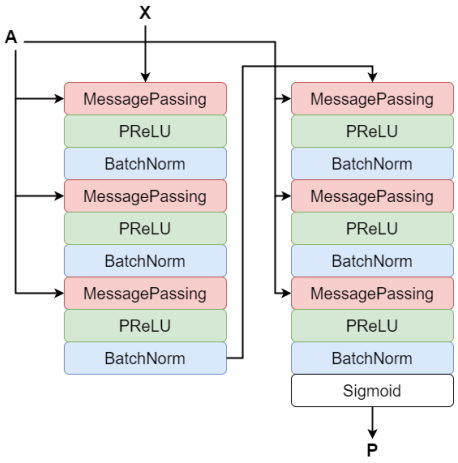


Fig. 7. The explainer network consists of six message-passing blocks, followed by a sigmoid layer that generates a probability distribution over the nodes, reflecting their importance for fault probability prediction.

using mini-batches of 32 service meshes. Network parameters were optimized using the Adam optimizer [40] with a learning rate of 2e-4. The mean squared error (MSE) served as the loss function. After training, we restored the parameters of the best performing model on the validation set.

C. GNN Training for Identifying Critical Components

After developing a highly accurate GNN model for fault prediction, we turned our attention to understanding how specific parts of the service mesh influence the predicted system-level fault probability. In each network, our goal is to separate the network into two complementary parts: one containing at most B% of the nodes and preserving prediction accuracy (the relevant subgraph), and one where the prediction accuracy degrades significantly (the irrelevant subgraph). This way, the selected nodes are both necessary and sufficient for the prediction. Exhaustively searching for such subnetworks by evaluating all possible node subsets is computationally infeasible, which motivates the need for a learned masking mechanism.

To address this challenge, we designed an explainability algorithm that assigns importance scores to the nodes of a network. Drawing inspiration from previous methods like L2X [41], INVASE [42], and PGExplainer [43], our approach uses an explainer neural network to compute node importance probabilities via gradient descent. Based on these probabilities, we can select the most important $B\%$ of the nodes in the service mesh with respect to fault prediction, providing a practical solution to our problem that minimizes prediction error.

Our algorithm, depicted in Figure 6, consists of two neural networks: an explainer GNN and a predictor GNN. The explainer GNN generates a Bernoulli distribution over the nodes of the service mesh, as illustrated in Figure 7. The probabilities assigned to each node reflect its significance in shaping the fault prediction process. From this distribution, we can sample a node mask that selects the most important nodes. This mask is then applied to both the adjacency matrix $\mathcal{A}$ and the feature matrix $\mathcal{X}$ through element-wise multiplication, producing a masked version of the input graph. The predictor GNN, trained for fault prediction as described in the previous subsection, evaluates three versions of the graph: the original unmasked graph $\mathcal{G}$, the subgraph selected by the explainer after masking g, and its complementary subgraph $\bar{g}$ (the nodes not selected by the explainer). If the node mask captures the most relevant nodes within the service mesh, the predictor's performance on the selected subgraph g should closely approximate its performance on the original graph $\mathcal{G}$, while its performance on the complementary subgraph $\bar{g}$ should drop significantly.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

This demonstrates that the selected nodes are really the critical components driving the fault prediction: they are necessary because removing them significantly impacts the prediction, and sufficient because the subgraph alone can replicate the original prediction.

The explainer and predictor GNNs are trained in alternating steps. In each iteration:

- **Predictor update**: The explainer generates a probability distribution p over the nodes of the graph. A node mask m is then sampled by applying Bernoulli sampling to this probability distribution. This mask defines a selected subgraph g and its complementary subgraph $\bar{g}$. The predictor f is trained to minimize the mean squared error (MSE) between its predictions and the true system-level fault probability on both subgraphs:

$$\mathcal{L}_f = \text{MSE}(f(g), y) + \text{MSE}(f(\bar{g}), y)$$

- **Explainer update**: The predictor, with its updated parameters, is evaluated on the subgraphs g and $\bar{g}$ to compute the reward. The explainer s is then optimized using the REINFORCE algorithm [44] with the loss:

$$\mathcal{L}_s = R \cdot \text{BCE}(p, m)$$

  where the reward function R is based on the difference between the predictor's performance on the selected subgraph and its complementary subgraph, with an added term that encourages sparsity in the selected nodes [43]:

$$R = \text{MSE}(f(\bar{g}), y) - \text{MSE}(f(g), y) + \text{ReLU}(\mu_p - B)$$

  Here, μ_p is the mean of the probability distribution p, and B is the budget which controls the expected number of nodes selected by the explainer.

The dataset was split into training, validation, and test sets following the same procedure used for fault prediction, and the same hyperparameters were retained.

D. Computational Complexity

Our approach is computationally efficient, as both the fault probability estimation and the explainability algorithm rely on Graph Neural Networks (GNNs). The key property of GNNs is that they operate on sparse adjacency matrices, enabling efficient storage and computation. The computational complexity of message-passing layers in GNNs is O(E), where E is the number of edges in the graph. Since service meshes typically have far fewer edges than the theoretical $O(N^2)$ bound for dense graphs, this makes GNN-based fault probability estimation scalable even for large-scale systems.

Since our explainer network is implemented as another GNN that learns to generate node importance scores, its complexity remains the same as the fault probability predictor GNN. Its overhead comes from sampling node masks and computing the reward function, but these operations are lightweight compared to the forward and backward passes of a GNN.

V. Experiment 1: Fault prediction for Feature Enriched Service Meshes

In this experiment, we evaluate the performance of our GNN-based approach for fault prediction on two types of service meshes: tree-shaped service meshes and general directed acyclic graphs (DAGs). Fault prediction serves as a foundation for identifying critical components within the service mesh, as accurately predicting system fault probabilities allows us to analyze the impact of removing specific nodes. Tree-shaped meshes represent a simpler hierarchical topology, making it easier to trace service dependencies, whereas general DAGs capture more complex, non-hierarchical relationships between services, where identifying critical components might be more challenging. All experiments were repeated three times, and all results reported are the mean and standard deviation across these three runs.

A. Training Dataset

The generation of training datasets for the REST-based approach involves constructing synthetic models that simulate complex cloud architectures characterized by a service mesh pattern. The RESTful services are designed as nodes within a directed acyclic graph (DAG) structure, where the root node initiates requests that propagate through intermediary nodes to leaf nodes. Intermediary nodes handle requests probabilistically, relaying them towards child nodes or providing direct responses. Leaf nodes finalize interactions by returning HTTP status codes, either signifying successful operations (HTTP 2xx) or indicating server failures (HTTP 5xx), the latter of which triggers a defined retry mechanism. Each component's operations and interactions are probabilistically modulated, enabling varied simulations reflective of real-world scenarios. Utilizing the Python programming language, a domain-specific language (DSL) was devised to model these interactions within the RESTful framework. This automation facilitates the creation and execution of extensive simulations, with predefined ranges for critical parameters such as the number of items and buffer sizes, enhancing the robustness of the dataset. The training dataset generation method and the volume of generated samples ensure comprehensive coverage of potential states and transitions, thereby obviating the necessity for noise reduction. The framework's methodological approach ensures that the resultant dataset is both diverse and representative of potential real-world RESTful service interactions, significantly contributing to the accuracy and reliability of subsequent analyses and models.

The current approach prioritizes computational efficiency and simplifies the simulation of network dynamics by focusing failures predominantly at the leaf nodes. This decision ensures that the model remains scalable and manageable, particularly when simulating extensive service meshes with multiple layers of intermediate nodes. By using retry counters at intermediary nodes, the model effectively emulates the robustness strategies typical in contemporary cloud infrastructures, where fault tolerance is largely handled through retries and load balancing rather than expecting individual intermediaries to manage failure. This abstraction aligns with industry practices,

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

TABLE II
SAMPLE OF OBSERVABLE ACTION SIMULATION RESULTS

id	from	to	method	action	result
0	X0	X22	GET	req	
1	X22	X24	GET	req	
2	X24	X19	GET	req	
3	X19	X13	GET	req	
4	X13	X12	GET	req	
5	X12	X17	GET	req	
6	X12	X17	GET	resp	2xx
7	X13	X12	GET	resp	2xx
8	X19	X13	GET	resp	2xx
9	X24	X19	GET	resp	2xx
10	X22	X24	GET	resp	2xx
11	X0	X22	GET	resp	2xx
12	X0	X22	GET	req	
13	X22	X24	GET	req	
14	X24	X5	GET	req	
15	X24	X5	GET	resp	2xx
16	X22	X24	GET	resp	2xx
17	X0	X22	GET	resp	2xx
18	X0	X15	GET	req	
19	X15	X14	GET	req	
20	X14	X11	GET	req	
21	X14	X11	GET	resp	2xx
22	X15	X14	GET	resp	2xx
23	X0	X15	GET	resp	2xx
24	X0	X23	GET	req	
25	X0	X23	GET	resp	5xx

where resilience is achieved through redundancy and failover mechanisms, minimizing the need to simulate internal failures at all node levels within the architecture.

Training datasets were generated based on the approach described above. Several cases, each containing simulation datasets, were generated for different architecture topologies (based on random DAGs). These then were expanded with sets of sub-cases, each with random parameter values for $P_{fail}[m]$ and $retries[n]$ following a uniform distribution. For each case, a graph description in Graph Exchange XML Format [45] was augmented with these new node attributes.

As internal state graph transition state simulations were considered less important in this phase of research, their length was reduced to 10.000 steps. To aid experiments focusing on the external observable properties of the cloud applications, a new filtered state simulation format was devised, as illustrated by Table II. This message simulation dataset consists of the following fields:

- id: event identifier, the discrete timestamp of the logical clock
- $from$: message source node identifier in format X_n where is n is the node identifier
- to: message destination node identifier in format X_n
- $method$: simulated HTTP method, only GET is implemented at the time of writing
- $action$: simulated HTTP action, with values req for requests and $resp$ for responses
- $result$: simulated HTTP result code for $resp$ actions, with values $2xx$ for normal responses, and $5xx$ for error responses

Based on the core model properties and node strategies described in the previous section, the message patterns follow a depth-first walk pattern with optional back-tracking. This means that for any given pair of communicating nodes, the following statements hold true:

- for each req action there must be at least one $resp$ action, more $resp$ actions are allowed in case of errors
- for each req action at least one $resp$ action is mandatory, so requests are not queued

At this phase of research, the total experimental dataset volume has exceeded 2.1 GB, including cloud architecture models in 50 cases for tree-shaped service meshes and 10 cases for DAG-based service meshes, each with distinct topologies. Each tree-shaped case contains 50 subcases, and each DAG-based case contains 10 subcases. The graphs in the dataset consist of 25 RESTful nodes per service mesh. While the underlying graph structure remains the same within each case, the node features—the number of retries for intermediary nodes and the probability of failure for leaf nodes—vary across subcases, offering a diverse set of configurations for analysis. The calculated system-level fault probabilities in the dataset range from a minimum of 0.0 to a maximum of 1.0, with a median of 1.2e-5.

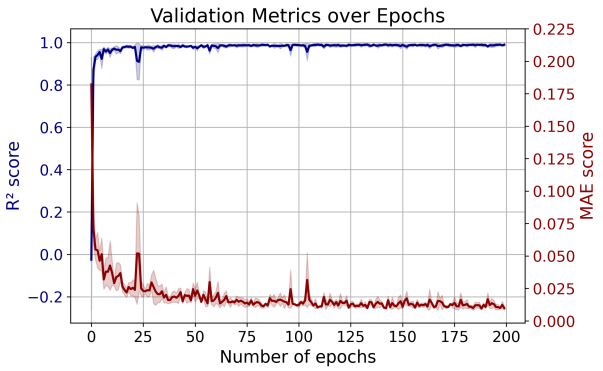


Fig. 8. Learning curve for fault probability prediction on tree-shaped service meshes: validation R² score (left axis) and validation MAE score (right axis) over epochs (mean ± standard deviation across 3 runs).

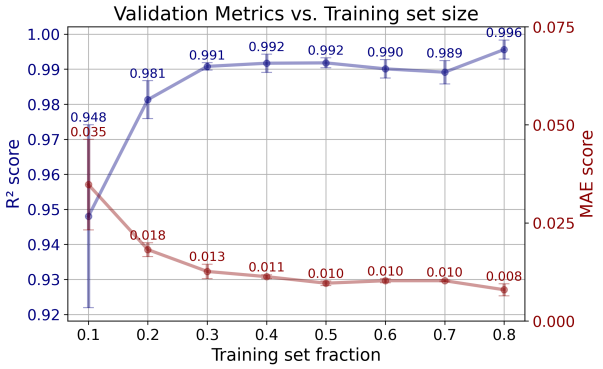


Fig. 9. Test R² score (left axis) and test MAE score (right axis) versus training set fraction for fault probability prediction on tree-shaped service meshes. Error bars represent mean ± standard deviation across 3 runs. Numerical labels indicate the mean value at each point.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

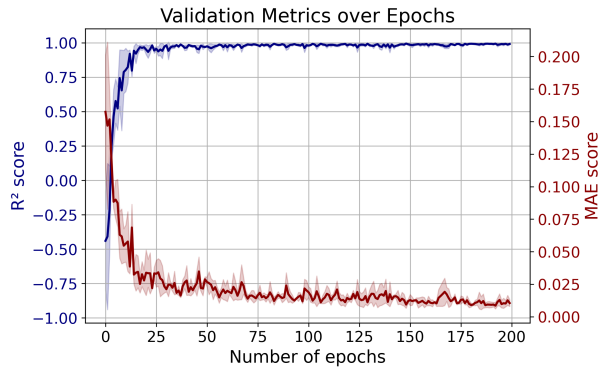


Fig. 10. Learning curve for fault probability prediction on tree-shaped service meshes: validation R^2 score (left axis) and validation MAE score (right axis) over epochs (mean ± standard deviation across 3 runs).

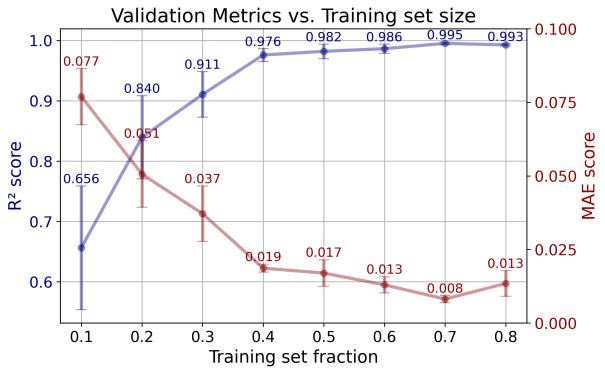


Fig. 11. Test R^2 score (left axis) and test MAE (right axis) versus training set fraction for fault probability prediction on DAG-shaped service meshes. Error bars represent mean ± standard deviation across 3 runs. Numerical labels indicate the mean value at each point.

B. Fault Prediction Results on Tree-shaped Service Meshes

In this subsection, we present the results of our experiments on tree-shaped service meshes. These experiments demonstrate the model's ability to capture both the relationships between nodes and their connections, as well as how variations in node features and the overall mesh structure influence system reliability.

As shown in Figure 8, the learning curve indicates effective training, with a consistent decrease in the MAE score and a corresponding increase in the validation R^2 score over epochs. The best model on the validation set achieved a MAE of 0.009 ± 0.001 on the test set, demonstrating strong generalization across diverse tree-shaped service mesh topologies. The regression plot further highlights the close alignment between predicted and actual system fault probabilities, with a test R^2 score of 0.992 ± 0.005 (Figure 12, subfigure a)).

Additionally, we examined the effect of varying the training set size on model performance. As shown in Figure 9, the plot presents the test R^2 score and test MAE score as functions of the training set fraction. The figure shows that the test MAE decreases consistently as the training set size increases, while the test R^2 score correspondingly improves and stabilizes at higher values. These results highlight the importance of using a large and diverse dataset to enable the model to effectively capture the underlying dependencies within the service mesh. This is crucial for further analyses on identifying critical components within the network.

C. Fault Prediction Results on DAG-shaped Service Meshes

In this subsection, we extend our experiments to DAG-shaped service meshes to evaluate the model's ability to generalize across more complex network topologies.

As shown in Figure 10, the learning curve for DAG-shaped service meshes shows a pattern similar to that observed for tree-shaped meshes, with a consistent decrease in the MAE score and a corresponding increase in the validation R^2 score over epochs. The best model on the validation set achieved a test MAE of 0.009 ± 0.002, indicating strong generalization across diverse DAG-shaped service mesh topologies. The regression plot further confirms the close alignment between predicted and actual system fault probabilities, with a test R^2 score of 0.996 ± 0.003 (Figure 15, subfigure (a))).

We further evaluated the model trained on DAG-shaped service meshes by testing it on the tree-shaped service mesh dataset to assess its ability to transfer across different topologies. The pretrained model achieved a test MAE score of 0.038 ± 0.006 and a test R^2 score of 0.938 ± 0.014 on the tree-shaped service mesh test set. These results, while slightly lower than those of the model trained directly on tree-shaped meshes, clearly demonstrate the model's capacity to generalize effectively across different service mesh structures.

As in our experiments on tree-shaped service meshes, we explored the impact of varying the training set size on the model's performance. Figure 11 illustrates a clear trend: increasing the training set size leads to a decreasing MAE score and an increasing R^2 score on the test set, with both metrics stabilizing at higher values. This consistency affirms our earlier conclusion that larger, more diverse datasets allow the model to better capture the relationships within the service mesh topology.

D. Baseline Comparison

To contextualize the performance of our GNN model and to isolate the contributions of nonlinear transformations and message passing, we implemented a systematic set of baseline models:

- **LinReg:** We bucketized the node feature values into five buckets per feature by sorting the values and dividing them into equal-sized groups. The mean within each bucket was computed, resulting in a permutation-invariant representation that is independent of node ordering and robust to variations in graph size. A linear regression model was then fit to these aggregated features.
- **LinReg with spectral embeddings:** Since linear regression cannot capture the graph structure, we computed the spectral embedding of each graph, applied the same bucketization to the spectral coordinates, and concatenated these with the bucketized node features. A linear regression model was fit to the combined features.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

TABLE III
TEST SET MAE FOR BASELINE MODELS AND THE GNN ON TREE- AND DAG-SHAPED SERVICE MESHES.

Model	Tree MAE	DAG MAE
LinReg	0.218 ± 0.004	0.130 ± 0.017
LinReg with spectral emb.	0.135 ± 0.004	0.083 ± 0.018
MLP	0.177 ± 0.001	0.127 ± 0.020
MLP with spectral emb.	0.031 ± 0.001	0.025 ± 0.003
GNN	0.009 ± 0.001	0.009 ± 0.002

- **MLP:** This model uses the same architecture as the GNN, but with all message passing layers replaced by linear layers. Permutation invariance is preserved through global mean pooling, while the model loses the GNN's ability to capture node interactions through message passing.
- **MLP with spectral embeddings:** The same architecture as above, but with spectral coordinates concatenated to the node features. This provides information about the graph structure in the absence of message passing.

Table III presents the test set MAE for each model on tree- and DAG-shaped service meshes. The linear regression models achieve limited accuracy, even when spectral embeddings are included, showing their inability to capture complex interactions between nodes. The MLP baselines provide improvements, particularly when spectral embeddings are used, but still fall short of the GNN's performance. These results highlight the critical role of message passing in effectively modeling node interactions. At the same time, they illustrate the tradeoff between model complexity and interpretability, motivating the explainability method introduced in the next section.

VI. EXPERIMENT 2: DETECTION OF CRITICAL COMPONENTS WITH GNN EXPLAINABILITY

In this experiment, we focus on our core objective: identifying the critical components within service meshes using GNN explainability techniques. By applying an explainability method to a well-trained GNN model, we determine which nodes (or subnetworks) have the most significant influence on the system's overall reliability. This allows us to detect both weak points and essential nodes whose failure or underperformance could severely impact service quality. This deeper insight is crucial for improving service robustness, as it enables targeted troubleshooting in the network's most influential areas. Additionally, this understanding lays the groundwork for future work, where the cloud debugger system can be steered toward potential failure points, enabling more effective remediation and resilience testing.

A. Training Dataset

For this experiment, we used the same datasets as in the fault prediction experiments, which consist of both tree-shaped and DAG-shaped service meshes. These datasets allow us to evaluate the model's ability to detect critical components across varying topologies and service configurations.

B. Explainability Results on Tree-shaped Service Meshes

In this section, we present the explainability results for tree-shaped service meshes. Using the predictor GNN, we applied our explainability algorithm to identify which nodes are most influential in predicting system fault situations, discovering the critical components of the networks.

Figure 12 shows the regression results for the predictor GNN on tree-shaped service meshes. The first plot (a) illustrates the predictions made using all nodes in the service mesh, which align very closely with the actual system fault probabilities, achieving an R^2 score of 0.992 ± 0.005 on the test set. This result indicates the model's strong predictive performance when using the full set of nodes in the network. Plot (b) demonstrates the predictions made using only the top 30% of nodes selected by the explainer network. Despite reducing the input to a smaller subset, the model maintains strong performance, achieving an R^2 score of 0.974 ± 0.018. This is significant, as it confirms that these selected nodes explain most of the variance in the predicted fault outcomes, effectively identifying them as the most important components for prediction. In contrast, plot (c) shows the predictions made on the complementary 70% of nodes, where the R^2 score drops to 0.104 ± 0.042. This suggests that these nodes provide minimal information about the fault outcomes, essentially leading to a baseline mean prediction. The dramatic drop in performance further reinforces the importance of the selected 30% of nodes in accurately predicting the network's reliability.

These results show how well our explainability method works, with the selected nodes being key to predicting fault outcomes accurately. The sharp drop between the predictive power of the selected and unselected nodes affirms the power of our method in identifying key components that drive network reliability.

We further investigated how adjusting the selection budget, i.e., the proportion of selected nodes, influences the model's predictive performance. By varying the budget from 10% to 50%, clear patterns emerge in both the R^2 and MAE scores for the selected and complementary node sets. As shown in Figure 13, the R^2 score for the selected nodes remains consistently high, between 0.991 and 0.996, demonstrating that even small subsets of nodes retain most of the critical information required for accurate fault prediction. In contrast, the R^2 score for the complementary nodes steadily decreases from 0.306 at 10% budget to 0.048 at 50%, as increasingly informative nodes are moved to the selected set. A similar trend is seen in the MAE scores: the error for the selected nodes remains low, between 0.014 and 0.018, while the complementary nodes exhibit rising errors, from 0.176 to 0.255. These findings clearly highlight the strength of our explainability method: the selected nodes consistently concentrate the predictive power, while the complementary nodes contribute little to the model's fault prediction accuracy.

To demonstrate better how our explainer network works, we show a simple example in Figure 14 (a). In this example, nodes are colored based on their selection probabilities, which show how important each node is for predicting the fault outcome. There is a leaf node next to the root with the relatively high

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

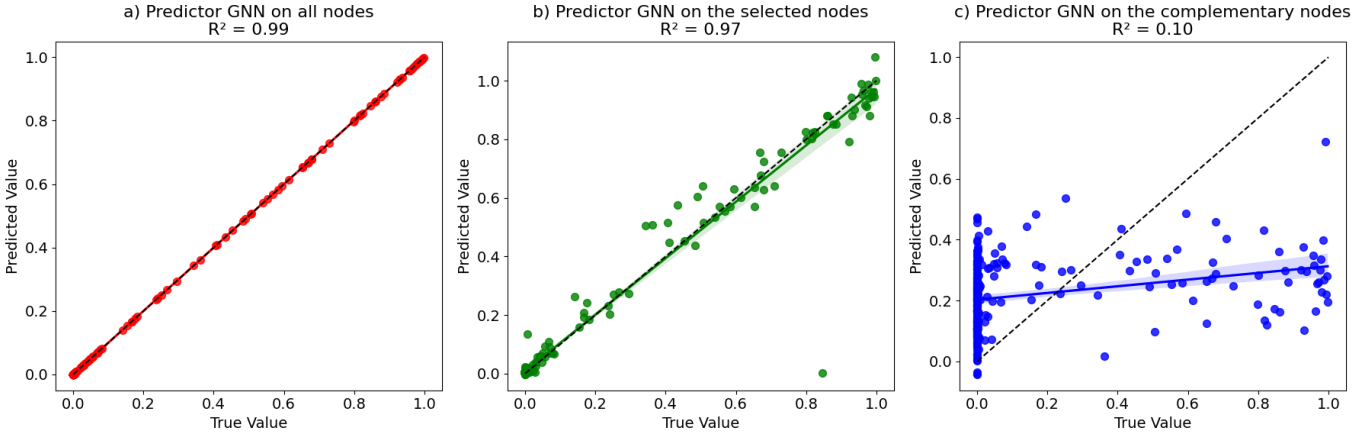


Fig. 12. Regression plots for fault probability prediction on tree-shaped service meshes using the predictor GNN: a) Predictions on 100% of the nodes, closely aligned with the ground truth values, achieving an R^2 score of 0.992, indicating excellent predictive performance; b) Predictions using the selected 30% of nodes (chosen by the explainer network), maintaining strong performance with an R^2 score of 0.974, confirming that these nodes are critical for predicting system-level fault probabilities and capturing most of the variance in the system; c) Predictions using the complementary 70% of unselected nodes, with an R^2 score of 0.104, demonstrating that these nodes provide little information about the system-level fault probabilities, essentially resulting in a baseline mean prediction. This clearly demonstrates the algorithm's success in identifying critical components, as the selected nodes retain most of the variance while the unselected ones provide minimal predictive value.

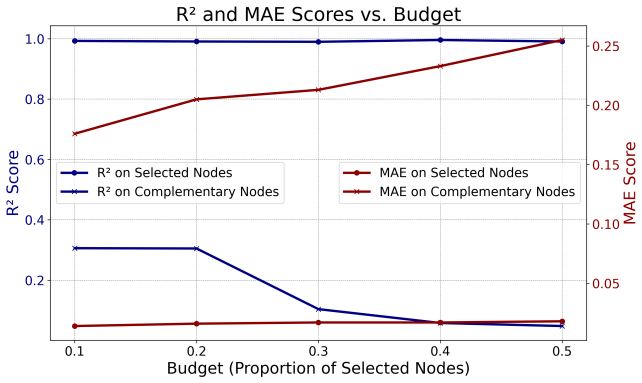


Fig. 13. R^2 and MAE Scores vs. Budget (Proportion of Selected Nodes): As the selection budget increases from 10% to 50%, the R^2 score for selected nodes remains consistently high (between 0.991 and 0.996), demonstrating that even small subsets capture most of the predictive information. In contrast, the R^2 score for complementary nodes steadily decreases from 0.306 to 0.048 as more informative nodes are moved to the selected set. Similarly, the MAE score for selected nodes remains low, while the MAE score for complementary nodes increases, reinforcing the importance of the selected nodes in system fault probability prediction.

failure probability of 0.27 (marked in red). Since this node is close to the root and has a high risk of failure, it plays a key role in determining the overall system fault probability. As a result, the ground truth system failure probability is a high value of 0.93. Our explainer network correctly identifies this node as important, assigning it a high selection probability. Using only the selected nodes, the model predicts that system fault probability is 0.94, which is very close to the actual value. In contrast, when the predictor uses only the unselected nodes, the predicted system fault probability is 0.02. This large difference shows that the unselected nodes cannot be used to determine the fault. Most of the unselected nodes are stable, and the leaves that could fail are far from the root, where intermediary nodes can mitigate if something goes wrong.

In Figure 14 (b), we changed the example slightly by setting the failure probability of this critical leaf node next to the root to 0.0. This leads to a significant change in the predicted system fault probability. Using the selected nodes, the predicted system fault probability drops to 0.0 because the critical node no longer has a risk of failure. The predicted system fault probability for the unselected nodes stays the same. This result is what we would expect and confirms that the explainer works correctly.

In Figure 14 (c), we made another slight change by moving the critical leaf node to a lower level in the service mesh, attaching it to an intermediary node that can retry once after a failure. This change affects how the explainer behaves: now only this critical node gets selected. As we expected, the predicted system fault probability drops to 0.01 due to the recovery capability of the intermediary node, which can retry the request if a failure occurs.

C. Explainability Results on DAG-shaped Service Meshes

In this subsection, we extend our explainability analysis to DAG-shaped service meshes. As with the tree-shaped meshes, we used the predictor GNN and applied the explainability algorithm to determine which nodes were most critical for predicting the system fault outcomes.

Figure 15 presents the regression results for the DAG-shaped service meshes. In plot (a), we see the predictions made using all nodes in the network. The predictor closely aligns with the true system fault probabilities, achieving an R^2 score of 0.996 ± 0.003, showing that the model performs exceptionally well when considering the entire set of nodes. Plot (b) shows the predictions made using only the top 30% of nodes identified by the explainer network. The model maintains a strong performance, with an R^2 score of 0.861 ± 0.054, suggesting that these selected nodes capture most of the critical

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

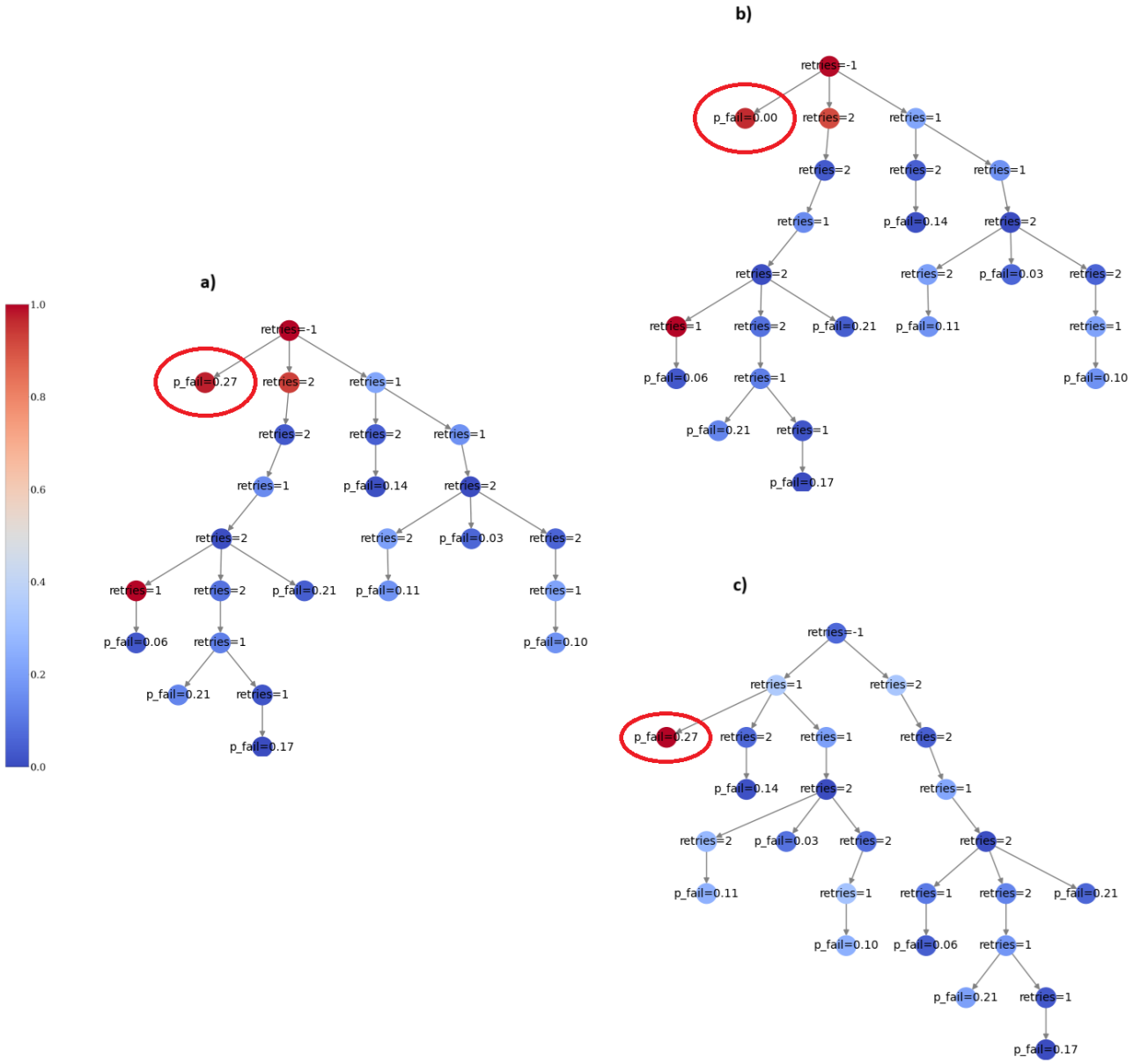


Fig. 14. Demonstration of explainer network on a service mesh. (a) A critical leaf node with high 0.27 failure probability near the root leads to a predicted system fault probability of 0.94, closely matching the ground truth of 0.93. (b) Setting the failure probability of the critical node to 0.00 reduces the predicted system fault probability to 0.00. (c) Moving the critical node lower in the tree, the predicted system fault probability drops to 0.01 due to the intermediary node's recovery ability.

information for fault prediction. In contrast, plot (c) shows the predictions made using the complementary 70% of nodes. The R^2 score here drops to 0.072 ± 0.086. This sharp decline highlights that these unselected nodes provide little to no useful information for predicting the fault outcomes. These results for DAG-shaped meshes align well with those from the tree-shaped meshes, which reinforces the effectiveness of our explainability method in identifying the critical components across different network structures.

However, in a small number of cases, we observe a discrepancy where the predicted system fault probability is close to zero while the true probability is significantly higher. This typically occurs when the explainer masks a few critical nodes that have a disproportionately large influence on the overall prediction. When these nodes are excluded from the selected subgraph, the model underestimates the system-level fault probability. These occasional failures suggest that the explainer may sometimes overlook sparse but highly informative components, and could benefit from further fine-tuning or regularization to increase robustness.

VII. Discussion and Conclusion

Designing dependable and scalable cloud services requires systematic tools to analyze complex behaviors and identify

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

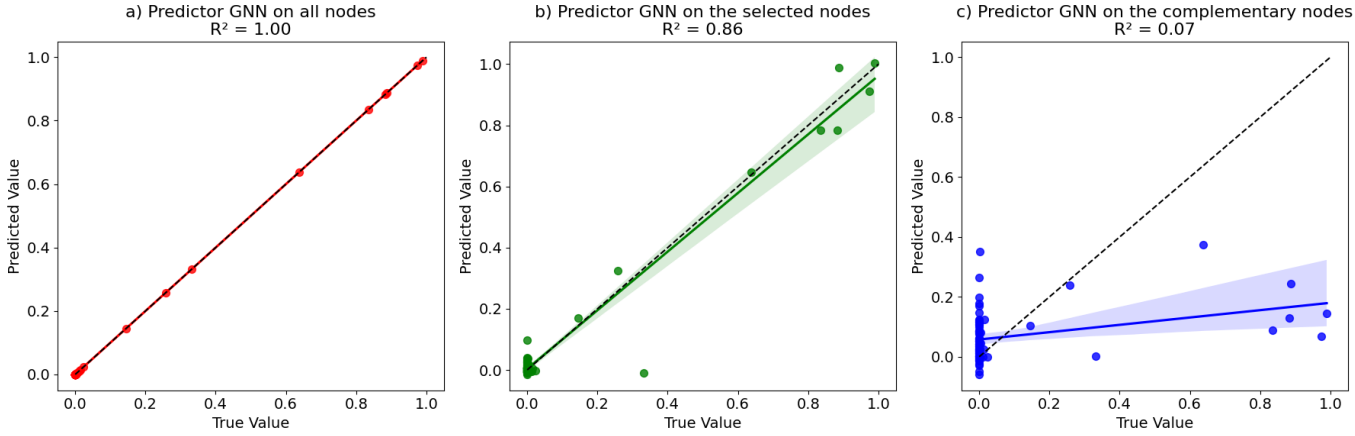


Fig. 15. Regression plots for fault probability prediction on DAG-shaped service meshes using the predictor GNN: (a) Predictions on 100% of the nodes, achieving an R^2 score of 0.996, indicating excellent predictive performance; (b) Predictions using the selected 30% of nodes (chosen by the explainer network), achieving an R^2 score of 0.861, showing that these nodes capture most of the critical information for fault probability prediction; (c) Predictions using the complementary 70% of nodes, achieving an R^2 score of 0.072, demonstrating that these nodes provide little to no value for predicting fault probabilities.

faults efficiently. While many existing approaches in the literature target specific aspects of fault detection—such as networking faults, security breaches, or system-level anomalies—they often operate at a fixed abstraction level and lack generalizability across cloud environments. This work contributes a unified and explainable approach to fault forecasting in microservice-based cloud systems, using probabilistic modeling and graph-based machine learning.

Our proposed method combines a formally verified simulation environment with data-driven fault prediction and explainability. The use of the PRISM model checker enables us to simulate fault propagation across a variety of service mesh topologies, generating training data without requiring deployment on live infrastructure. Starting from simple tree-structured graphs and extending to general DAGs, we demonstrated how the framework scales across increasing architectural complexity.

Graph Neural Networks allow us to capture both structural and feature-based dependencies in microservice interactions, achieving accurate system-level fault probability predictions. Furthermore, by integrating an explainability mechanism, the framework can highlight the most failure-prone microservices—guiding debugging efforts toward critical components and improving system-level fault understanding.

Although the current evaluation is based on synthetic datasets to ensure full control and formal validation, real-world applicability remains a priority. Future work will focus on integrating real-world cloud traces, adapting to dynamic operational environments, and extending the method to broader observability contexts.

The dataset, code, and additional materials related to this work are publicly available at: https://github.com/BME-SmartLab/GNN-Fault-Forecasting

ACKNOWLEDGMENT

The work reported in this paper, carried out as a collaboration between HUN-REN SZTAKI and BME, has been partly supported by the European Union project RRF-2.3.1-21-2022-00004 within the framework of the Artificial Intelligence National Laboratory. This work was partially funded by the National Research, Development and Innovation Office (NKFIH) under OTKA Grant Agreement No. K 132838. The presented work of R. Lovas was also supported by the János Bolyai Research Scholarship of the Hungarian Academy of Sciences. On behalf of the 'MILAB - SmartLab' cloud project, we are grateful for the possibility to use HUN-REN Cloud [46] which helped us achieve the results published in this paper.

◇ WordTune, Grammarly, OpenAI ChatGPT GPT-4: grammar enhancement. In case of ChatGPT, the prompt was *'You are a researcher, produce high-quality scientific text. Rewrite the following text to improve its grammar and scientific language: INPUT TEXT'* alike.

⋆ OpenAI ChatGPT GPT-4, Scite Assistant: used to create a draft, which was completed, rewritten and enhanced by the authors. The prompt was *'You are a researcher, produce high-quality scientific text. Write a summary draft on the following topics: INPUT TEXT'* alike.

REFERENCES

[1] David Jaramillo, Duy V Nguyen, and Robert Smart. Leveraging microservices architecture by using docker technology. In *SoutheastCon 2016*, pages 1–5, 2016.

[2] Pekka Pääkkönen and Daniel Pakkala. Reference architecture and classification of technologies, products and services for big data systems. *Big Data Res.*, 2(4):166–186, dec 2015.

[3] Attila Csaba Marosi, Márk Emődi, Attila Farkas, Róbert Lovas, Richárd Beregi, Gianfranco Pedone, Balázs Németh, and Péter Gáspár. Toward reference architectures: A cloud-agnostic data analytics platform empowering autonomous systems. *IEEE Access*, 10:60658–60673, 2022.

[4] Microsoft azure documentation - reference architectures. https://docs.microsoft.com/en-us/azure/architecture/browse/. Accessed: 2022-02-20.

[5] The TOGAF Standard, Version 9.2 Overview. https://www.opengroup.org/togaf. Accessed: 2022-02-12.

[6] What is a reference architecture? – enterprise it definitions. https://www.hpe.com/us/en/what-is/reference-architecture.html. Accessed: 2022-02-20.

[7] Wubin Li, Yves Lemieux, Jing Gao, Zhuofeng Zhao, and Yanbo Han. Service mesh: Challenges, state of the art, and future research opportunities. In *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, pages 122–1225, 2019.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223

[8] Jose Cunha, Joao Lourenco, and Vitor Duarte. *Debugging of parallel and distributed programs*, pages 101–136. Nova Science Publishers, 2001.

[9] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering*, 47(2):243–260, 2021.

[10] Eman Aljoumah, Fajer Al-Mousawi, Imtiaz Ahmad, Maha Al-Shammri, and Zahraa Al-Jady. Sla in cloud computing architectures: A comprehensive study. *Int. J. Grid Distrib. Comput*, 8(5):7–32, 2015.

[11] Bence Ligetfalvi, Márk Emődi, József Kovács, and Róbert Lovas. Fundamentals of a novel debugging mechanism for orchestrated cloud infrastructures with macrosteps and active control. *Electronics*, 10(24), 2021.

[12] Róbert Lovas, Ernö Rigó, Dániel Unyi, and Bálint Gyires-Tóth. Experiences with deep learning enhanced steering mechanisms for debugging of fundamental cloud services. *IEEE Access*, 11:26403–26418, 2023.

[13] Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of Probabilistic Real-Time Systems. In Ganesh Gopalakrishnan and Shaz Qadeer, editors, *Computer Aided Verification*, volume 6806, pages 585–591. Springer Berlin Heidelberg, Berlin, Heidelberg, 2011.

[14] József Kovács, Bence Ligetfalvi, and Róbert Lovas. Automated debugging mechanisms for orchestrated cloud infrastructures with active control and global evaluation. *IEEE Access*, 12:143193–143214, 2024.

[15] Giuseppe Aceto, Alessio Botta, Walter De Donato, and Antonio Pescapè. Cloud monitoring: Definitions, issues and future directions. In *2012 IEEE 1st International Conference on Cloud Networking (CLOUDNET)*, pages 63–67. IEEE, 2012.

[16] Sheikh Umar Farooq, SMK Quadri, and Nesar Ahmad. Metrics, models and measurements in software reliability. In *2012 IEEE 10th international symposium on applied machine intelligence and informatics (SAMI)*, pages 441–449. IEEE, 2012.

[17] Sara Zatout, Maya Souilah Benabdelhafid, and Mahmoud Boufaida. Formal Transaction Modeling and Verification for an Adaptable Web Service Orchestration. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, pages 531–536, July 2018.

[18] Alireza Souri, Amir Masoud Rahmani, and Nima Jafari Navimipour. Formal verification approaches in the web service composition: A comprehensive analysis of the current challenges for future research. *International Journal of Communication Systems*, 31(17):e3808, 2018. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/dac.3808.

[19] Mounya Smara, Makhlouf Aliouat, Al-Sakib Khan Pathan, and Zibouda Aliouat. Acceptance test for fault detection in component-based cloud computing and systems. *Future Generation Computer Systems*, 70:74–93, 2017.

[20] Aaqif Abbasi, Shahab Band, Mohammed A. A. Al-qaness, Almas Abbasi, Nahsat Al-Jallad, and Amir Mosavi. Resource-aware network topology management framework. *Acta Polytechnica Hungarica*, 02 2020.

[21] Rupesh Raj Karn, Rammi Das, Dibakar Raj Pant, Jukka Heikkonen, and Rajeev Kanth. Automated testing and resilience of microservice's network-link using istio service mesh. In *2022 31st Conference of Open Innovations Association (FRUCT)*, pages 79–88, 2022.

[22] Qing Cao and Haoran Niu. Higher-order markov graph based bug detection in cloud-based deployments. In *2022 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pages 153–160, 2022.

[23] Valeria Pontillo, Maarten Vandercammen, Sarah Verbelen, and Coen De Roover. Ferrari: Failure reproduction through automatic test case generation and stack trace analysis. In *BENEVOL24: The 23rd Belgium-Netherlands Software Evolution Workshop*, pages 1–13. CEUR Workshop Proceedings, 2024. BENEVOL 2024 : The 23rd Belgium-Netherlands Software Evolution Workshop ; Conference date: 21-11-2024 Through 22-11-2024.

[24] PeiYun Zhang, Sheng Shu, and MengChu Zhou. An online fault detection model and strategies based on svm-grid in clouds. *IEEE/CAA Journal of Automatica Sinica*, 5(2):445–456, 2018.

[25] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. Latent error prediction and fault localization for microservice applications by learning from system trace logs. In *27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 26–30, 2019.

[26] Fanfei Meng, Lalita Jagadeesan, and Marina Thottan. Model-based reinforcement learning for service mesh fault resiliency in a web application-level, 2021.

[27] Raphael Rouf, Mohammadreza Rasolroveicy, Marin Litoiu, Seema Nagar, Prateeti Mohapatra, Pranjal Gupta, and Ian Watts. Instantops: A joint approach to system failure prediction and root cause identification in microservices cloud-native applications. In *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*, ICPE '24, page 119–129, New York, NY, USA, 2024. Association for Computing Machinery.

[28] Angelos-Christos Maroudis, Theodoros Theodoropoulos, John Violos, Aris Leivadeas, and Konstantinos Tserpes. Leveraging graph neural networks for sla violation prediction in cloud computing. *IEEE Transactions on Network and Service Management*, 2023.

[29] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. Microrca: Root cause localization of performance issues in microservices. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9, 2020.

[30] Dongjie Wang, Zhengzhang Chen, Jingchao Ni, Liang Tong, Zheng Wang, Yanjie Fu, and Haifeng Chen. Interdependent causal networks for root cause localization. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, page 5051–5060, New York, NY, USA, 2023. Association for Computing Machinery.

[31] Yu Gan, Guiyang Liu, Xin Zhang, Qi Zhou, Jiesheng Wu, and Jiangwei Jiang. Sleuth: A trace-based root cause analysis system for large-scale microservices with graph neural networks. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ASPLOS '23, page 324–337, New York, NY, USA, 2024. Association for Computing Machinery.

[32] Farida Asadova, Gábor Kertész, Róbert Lovas, and Sándor Szénási. Fault detection in gpu-enabled cloud systems – an overview. In *2022 IEEE 20th Jubilee World Symposium on Applied Machine Intelligence and Informatics (SAMI)*, pages 000317–000322, 2022.

[33] Andrea Bianco and Luca de Alfaro. Model checking of probabilistic and nondeterministic systems. In *International Conference on Foundations of Software Technology and Theoretical Computer Science*, pages 499–513. Springer, 1995.

[34] D.C. Verma. Service level agreements on IP networks. *Proceedings of the IEEE*, 92(9):1382–1388, 2004.

[35] Philipp Wieder, Joe M. Butler, Wolfgang Theilmann, and Ramin Yahyapour. *Service Level Agreements for Cloud Computing*. Springer Science & Business Media, November 2011.

[36] Wubin Li, Yves Lemieux, Jing Gao, Zhuofeng Zhao, and Yanbo Han. Service mesh: Challenges, state of the art, and future research opportunities. In *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, pages 122–1225. IEEE, 2019.

[37] Leonard Richardson and Sam Ruby. *RESTful Web Services*. " O'Reilly Media, Inc.", 2008.

[38] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.

[39] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.

[40] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *arXiv preprint arXiv:1412.6980*, 2014.

[41] Jianbo Chen, Le Song, Martin Wainwright, and Michael Jordan. Learning to explain: An information-theoretic perspective on model interpretation. In *International Conference on Machine Learning*, pages 883–892. PMLR, 2018.

[42] Jinsung Yoon, James Jordon, and Mihaela van der Schaar. Invase: Instance-wise variable selection using neural networks. In *International Conference on Learning Representations*, 2019.

[43] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. Parameterized explainer for graph neural network. In *Advances in Neural Information Processing Systems*, volume 33, pages 19620–19631, 2020.

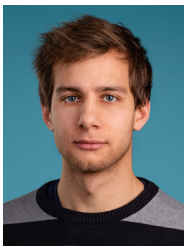
[44] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.

[45] GEXF File Format. https://gexf.net/. [Online] Accessed 28 September 2022.

[46] Mihály Héder, Ernő Rigó, Dóra Medgyesi, Róbert Lovas, Szabolcs Tenczer, Ferenc Török, Attila Farkas, Márk Emődi, József Kadlecsik, György Mező, Ádám Pintér, and Péter Kacsuk. The Past, Present and Future of the ELKH Cloud. *Információs Társadalom (Information Society)*, XXII(2), 2022.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Network and Service Management. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNSM.2025.3602223



Dániel Unyi is a PhD candidate in Computer Engineering at Budapest University of Technology and Economics. His research focuses on graph-based representation learning and explainable AI, with a particular emphasis on various applications. During his PhD, he has earned top placements in international data science competitions. In addition to his academic work, he is actively involved in several industry projects as a data scientist, primarily in the financial and forensic domains.



Ernö Rigó is the Head of the Department of Network Security and Internet Technologies at the Institute for Computer Science and Control (SZTAKI), Hungarian Research Network (HUN-REN). He received his MSc degree in systems engineering from the Budapest University of Technology and Economics (BME). He is an assistant lecturer at the Institute for Cyber-Physical Systems at the John von Neumann Faculty of Informatics, Óbuda University, where he is also a PhD student. As an ISACA Certified Information Systems Auditor (CISA) and ISC2 Certified Information Systems Security Professional (CISSP), his PhD research is focused on security evaluation of dynamic cloud infrastructures. He is responsible for system design, implementation and day to day operation of the Cloud research infrastructure of HUN-REN. His cloud and security related research achievements contribute to the recently launched Artificial Intelligence and Autonomous Systems National Laboratories.



Bálint Gyires-Tóth is an associate professor at BME. He conducts research on fundamental and applied machine learning since 2007. With his leadership, the first Hungarian hidden Markov-model based Text-To-Speech (TTS) system was introduced in 2008. He obtained his PhD degree with summa cum laude in January 2014. Since then, his primary research field is deep learning. His main research interests are sequential data modelling with deep learning, self-supervised learning and deep reinforcement learning. He also participates in applied deep learning projects, including time series modelling, anomaly detection, computer vision and conversational AI. He was involved in various successful research and commercial projects. In 2017 he was certified as NVidia Deep Learning Institute (DLI) Instructor and University Ambassador. His AI-related research achievements contribute to the Artificial Intelligence Systems National Laboratory.



Róbert Lovas is the deputy director of IT at the Institute for Computer Science and Control (SZTAKI), Hungarian Research Network (HUN-REN). He received his PhD degree in Informatics from the Budapest University of Technology and Economics (BME). He is a habilitated associate professor and the founder of the Institute for Cyber-Physical Systems at the John von Neumann Faculty of Informatics, Óbuda University, and a member of the Committee on Information Science at the Hungarian Academy of Sciences. His research and development experience in wide range of application fields of distributed and parallel systems has been gained in various global, EU and national collaborations with academic organizations, universities, and enterprises focusing on computational chemistry, numerical meteorological modelling, bioinformatics, agriculture, connected cars, and Industry 4.0. He has been coordinating EU FP7/H2020 projects, and the HUN-REN Cloud research infrastructure of HUN-REN. His latest cloud, big data, IoT and AI-related research achievements contribute to the recently launched Artificial Intelligence and Autonomous Systems National Laboratories.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/